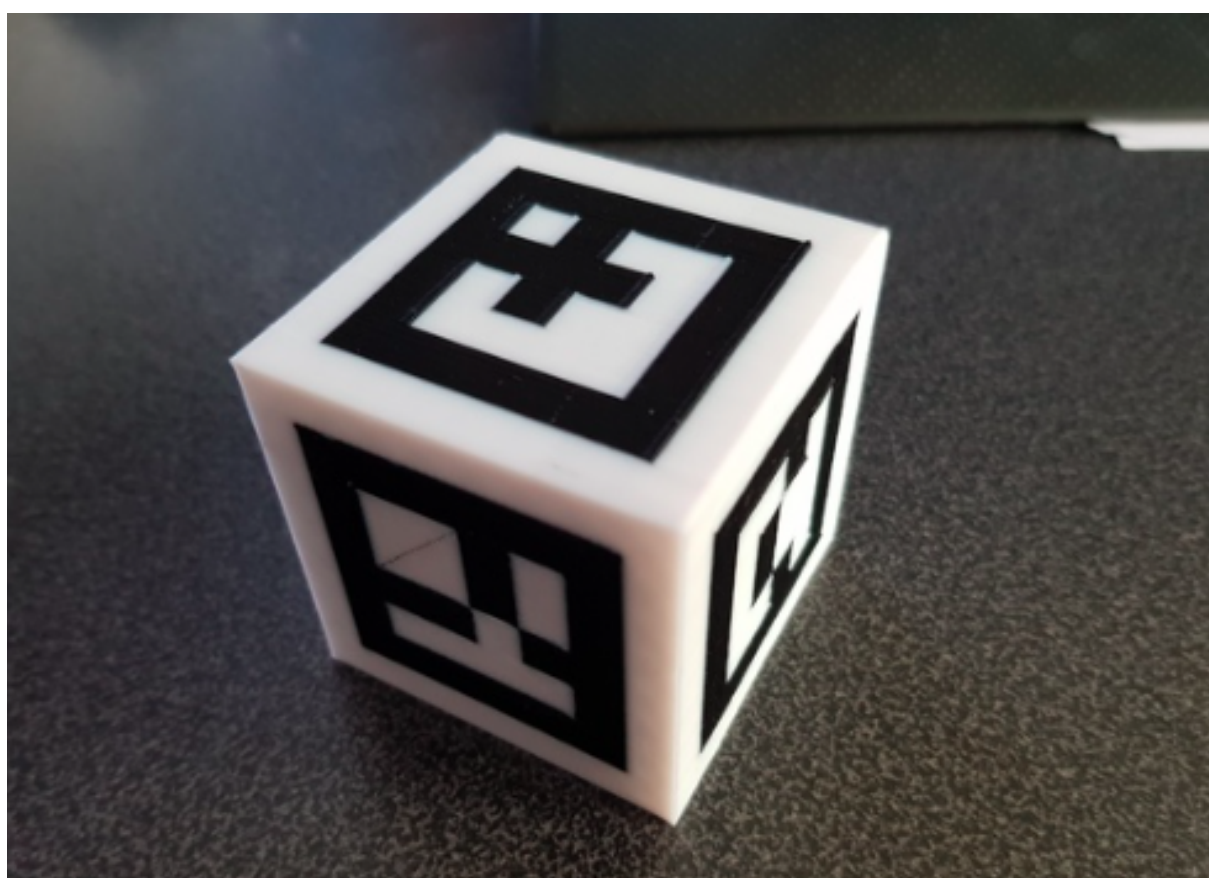


A Hands on lab from Aruco Generation to Augmented reality using opencv and c++



Submitted by:
Mir Mohibullah Sazid, Syma Afsha

Advisor: Dr. Josep Forest

Contents

Contents	I
1 Introduction	1
1.1 Lab 1: Introduction to OpenCV and Fiducial Markers	1
1.2 Lab 2: Generating and Utilizing ArUco Markers	1
1.3 Lab 3: Detection of ArUco Markers	1
1.4 Lab 4: Camera Calibration	1
1.5 Lab 5: Pose Estimation and Augmented Reality	2
2 Methodology and Results	3
2.1 Lab 1	3
2.2 Lab 2	3
2.3 Lab 3	4
2.4 Lab 4	5
2.5 Lab 5	6
3 Conclusion	9
4 Appendix	10

1 Introduction

This report outlines the structured learning experience conducted in our computer vision laboratory, which is divided into six distinct labs. Each lab is focused on a different aspect of utilizing OpenCV, a powerful library for computer vision tasks, with a specific emphasis on ArUco markers, a type of fiducial marker system. These markers are crucial for various computer vision applications, including object pose estimation, camera calibration, and augmented reality.

1.1 Lab 1: Introduction to OpenCV and Fiducial Markers

The first laboratory session involved installing OpenCV and learning the basics of fiducial markers. We explored how these markers could be used to detect an object's pose, including its position and orientation. Essential image processing techniques such as converting to grayscale, applying thresholds, blurring, detecting edges, and setting up interactive controls (trackbars) were covered to tweak processing settings and view results in real-time.

1.2 Lab 2: Generating and Utilizing ArUco Markers

In Lab 2, we focused on generating ArUco markers with unique IDs and constructing an ArUco board, used primarily for camera calibration and augmented reality applications. This session detailed the process of marker and board creation, which are essential for the accurate calibration of the camera.

1.3 Lab 3: Detection of ArUco Markers

Lab 3 was dedicated to the detection of ArUco markers, identifying them by their unique IDs. Various detection methods were examined, and the importance of marker size and alignment in achieving precise detection was emphasized.

1.4 Lab 4: Camera Calibration

In Lab 4, the camera was calibrated using the ArUco board to obtain the camera matrix and its distortion coefficients. This procedure is critical as it helps correct distortions in images captured by the camera, ensuring that the camera's specifications are accurate and the images are distortion-free.

1.5 Lab 5: Pose Estimation and Augmented Reality

Lab 5 involved estimating the pose of ArUco markers and building a wireframe cube around them. By using the camera's calibrated parameters and real images from the camera, we accurately determined the pose of the detected marker. This allowed us to visualize a three-dimensional cube around the marker, demonstrating the integration of digital image processing and physical space interactions.

Each lab session built upon the previous one, leading to a comprehensive understanding of the capabilities of OpenCV with ArUco markers for advanced computer vision applications. The skills and knowledge acquired are pivotal for anyone looking to explore further in the fields of augmented reality and robotic vision.

2 Methodology and Results

2.1 Lab 1

In this lab we utilize the OpenCV library for real-time video processing, specifically demonstrating edge detection via the Canny algorithm. By converting the video input to grayscale, applying Gaussian blurring, and then using Canny edge detection, it visually illustrates the contours and edges of objects captured by the webcam. The inclusion of trackbars allows users to dynamically adjust the threshold parameters, providing an interactive way to study how edge detection parameters influence the outcome. This setup is ideal for a lab focused on exploring image processing techniques and their practical applications in computer vision. The output of the program is given below.

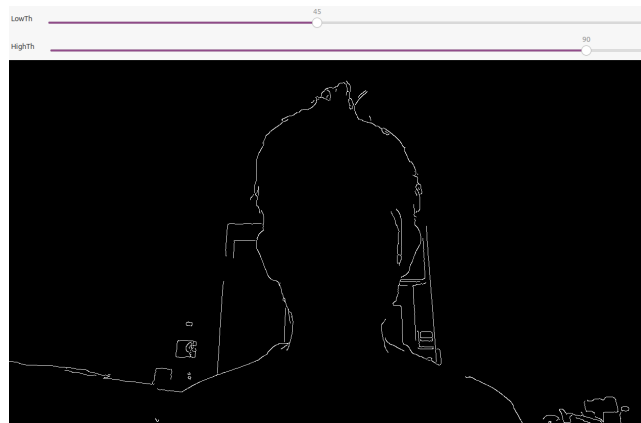


Figure 2.1: Canny Edge Image

2.2 Lab 2

This lab, utilizing OpenCV's ArUco library, is designed to generate ArUco markers, which are square binary patterns used for camera pose estimation and augmented reality applications. The program accepts command-line arguments to specify the ArUco dictionary, the marker's ID, its size, and the output file location. It includes a function, `getPredefinedDictionary`, to fetch the specified ArUco dictionary, ensuring robust error handling for undefined dictionaries. After generating the desired marker, the image is saved to a specified file. This setup is particularly useful for a lab focused on understanding marker-based tracking, illustrating practical applications in calibrating cameras and developing AR systems. By dynamically creating and visualizing different markers, we can explore how different parameters affect detection and tracking performance in computer vision systems. Here 2.2 is the output of the generated Aruco.

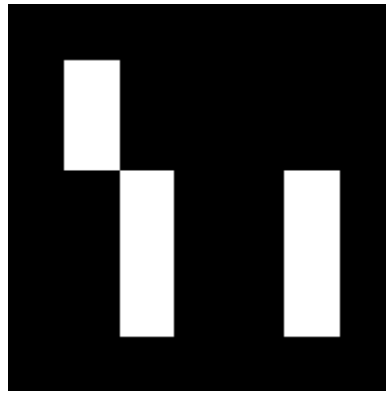


Figure 2.2: Generated Aruco

This this lab we use the OpenCV ArUco module to generate a grid board of ArUco markers, which are widely used in augmented reality and camera calibration tasks. The user specifies the number of rows and columns, the ArUco dictionary type, marker size, separation between markers, and the output file name via command-line arguments. The program we build, first retrieves the specified ArUco dictionary and then creates a grid board. It visualizes this board by generating an image file where each marker is distinctly separated, aiding in accurate camera calibration and pose estimation studies. This functionality is essential for creating controlled environments in computer vision experiments where precise spatial measurements are required. Successfully saving the generated board to an image file allows for reproducible setups in practical laboratory settings, making it a valuable tool for testing and demonstrating the effectiveness of various computer vision algorithms. Below 2.3 we can observe the output of the program.

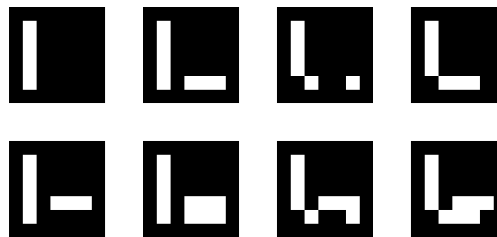


Figure 2.3: Generated Board

2.3 Lab 3

In this lab we used the OpenCV library's ArUco module to detect ArUco markers in real-time video captured from a webcam. The program initializes by setting up a video stream and continuously captures frames, within which it identifies and marks ArUco markers using a predefined dictionary `DICT_ARUCO_ORIGINAL`. Once markers are detected, the program highlights them by drawing boxes and marker IDs on the frame, displaying these annotations in a live window. This application is particularly useful for real-time tracking and augmented

reality scenarios, where recognizing and tracking fiducial markers are crucial. This program demonstrates effective implementation of marker detection which can be extended to study the behavior of tracking systems under various conditions, analyzing the robustness and precision of marker detection in dynamic environments. The output shows the detected marker in image 2.4



Figure 2.4: Detected Marker

2.4 Lab 4

In this lab, we have done camera calibration using opencv. The calibration process is crucial for computer vision applications to correct lens distortion and enhance the accuracy of metric measurements in the captured images. We have program it which starts by setting up command line parsing to accept various parameters essential for calibration, such as the grid dimensions of the ArUco marker board (number of squares in X and Y directions), the marker size, the separation between markers, and the ArUco dictionary type.

The process initiates by creating a video capture object to retrieve frames from a specified camera. As the program runs, it continuously captures video frames and detects ArUco markers in these frames. If markers are detected, the corners of these markers are stored along with their IDs for later use. The user can manually add frames to be used for calibration by pressing 'c'. The program emphasizes user interaction, guiding through on-screen instructions.

Once a sufficient number of frames are captured, the program proceeds to calibrate the camera. This involves calculating the camera matrix and distortion coefficients, which describe the camera's imaging characteristics. The calibration routine uses the positions of the detected markers to compute these parameters accurately, minimizing the reprojection error, which is a measure of the calibration accuracy.

After successful calibration, the results, including the camera parameters and the average reprojection error, are saved to an ymal output file. This file can be used for correcting distortions in future computer vision tasks using the same camera. This makes the program invaluable for applications in augmented reality, 3D reconstruction, and robotics where precise camera calibration is fundamental. The output informs the user of the calibration's efficacy and reli-

ability through the reported reprojection error, providing an empirical basis for assessing the calibrated camera's performance in real-world tasks.

The camera matrix we obtain is

$$\begin{bmatrix} 1.0098526442003833 \times 10^3 & 0 & 6.5557040839533522 \times 10^2 \\ 0 & 1.0065841955866184 \times 10^3 & 3.3960467688058378 \times 10^2 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.1)$$

The distortion coefficient is

$$\begin{bmatrix} 1.7688899541749847 \times 10^{-1} \\ -1.1516302915229100 \\ -3.7077171771476261 \times 10^{-3} \\ 2.3321746527067990 \times 10^{-3} \\ 2.9446158146633170 \end{bmatrix} \quad (2.2)$$

The average re projection error is: $3.7641900365392694 \times 10^1$

2.5 Lab 5

This lab we utilizes OpenCV's ArUco module to perform marker detection and pose estimation in a video stream captured from a camera. It first reads camera calibration parameters from a YAML file. Then, it continuously captures frames from the video stream, detects ArUco markers, and estimates their poses using the provided camera matrix and distortion coefficients. Detected markers are highlighted in the frame, and their translation and rotation vectors are printed. Additionally, the program draws coordinate axes on each marker to visualize its orientation in 3D space. The estimated pose of the marker is depicted by a Cartesian coordinate system, where the center of the marker acts as the origin. This coordinate framework comprises three orthogonal segments: a blue axis (Z-axis), perpendicular to the plane of the marker; a red axis (X-axis), aligned with one of the marker's sides; and a green axis (Y-axis), situated at a 90-degree angle from the X-axis within the marker's plane. These axes visually represent the orientation of the marker in 3D space. This application is valuable for various computer vision tasks such as augmented reality, object tracking, and robot navigation. The output of aruco pose is given below in 2.5:

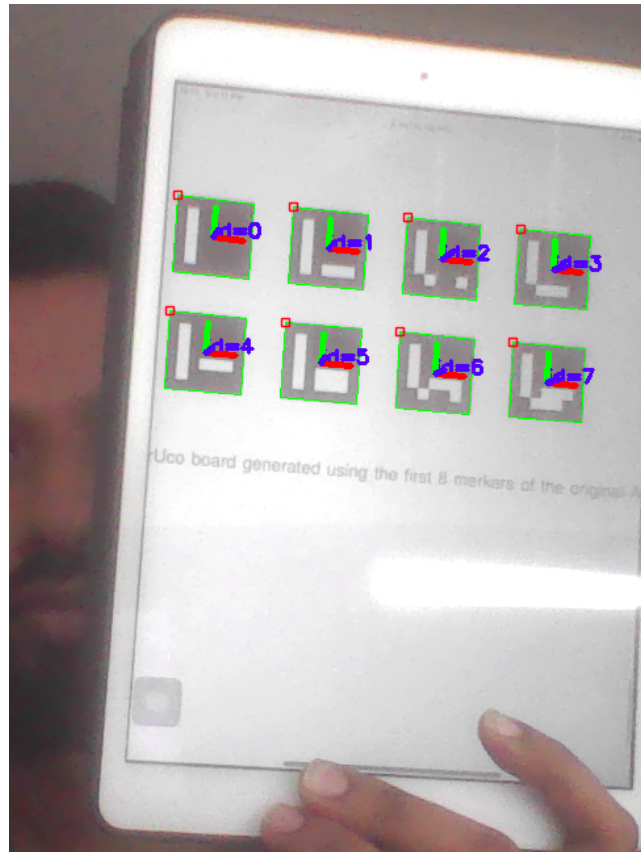


Figure 2.5: Pose of Aruco

This code detects ArUco markers in a video stream, estimates their pose, and draws wireframe cubes over the detected markers. It begins by parsing command-line arguments to specify the ArUco dictionary and the actual marker length. The video stream is then opened, and camera calibration parameters are loaded from a YAML file. For each frame, markers are detected, their pose is estimated, and wireframe cubes are drawn over them using the provided camera parameters. The drawn cubes represent the 3D position and orientation of the markers in the scene. Finally, the annotated frames are written to a video file. The wireframe cube is rendered using OpenCV's `line()` function to draw its edges. Positioned in 3D space relative to the marker's pose, the cube appears affixed to the marker within the camera-captured image. This outcome characterizes an augmented reality (AR) application, wherein a virtual 3D object is superimposed onto a physical object—in this instance, the ArUco marker. Through this technique, the marker's spatial information serves as a reference point, enabling the integration of virtual content into real-world environments, thus facilitating AR experiences and applications. This program can be useful for applications involving augmented reality, object tracking, and camera calibration, providing real-time visualization of ArUco markers' spatial information. The below image 2.6 shows the outcome of the result.

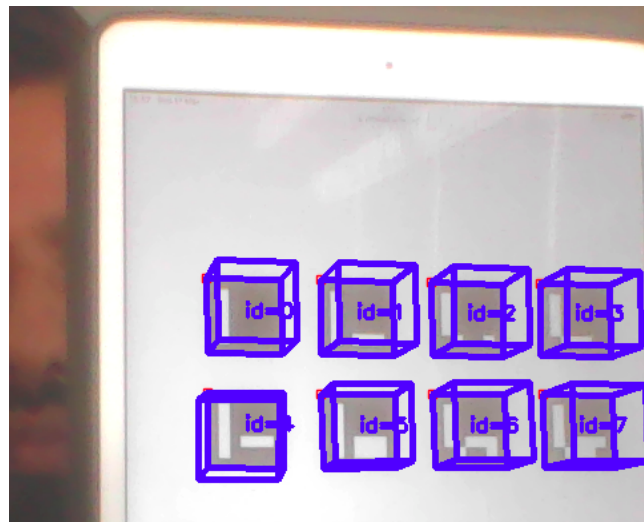


Figure 2.6: Wire-frame Cube using Aruco.

3 Conclusion

In conclusion, the series of labs showcased the diverse applications of computer vision techniques, particularly leveraging the capabilities of OpenCV. From basic edge detection to advanced marker detection and pose estimation, each lab demonstrated practical scenarios where computer vision plays a vital role. The ability to manipulate real-time video streams, identify objects, and estimate their poses opens doors to various fields, including augmented reality, robotics, and object tracking. These labs provide foundational knowledge and hands-on experience essential for understanding and implementing computer vision algorithms in real-world scenarios. Edge detection algorithms like Canny offer insights into image processing fundamentals, while marker-based tracking techniques, such as those demonstrated with ArUco markers, lay the groundwork for augmented reality applications. Additionally, camera calibration methodologies ensure accurate measurements and precise spatial understanding, crucial for tasks like 3D reconstruction and robotic navigation. Overall, the labs illustrate the transformative potential of computer vision technologies in revolutionizing industries ranging from healthcare to entertainment, offering solutions to complex problems and enhancing user experiences through immersive augmented reality environments, precise object tracking, and intelligent automation. With continued advancements in computer vision research and technology, the applications explored in these labs are poised to shape the future of various fields, driving innovation and transforming how we interact with the world around us.

Reference

[1] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.

4 Appendix

Please find the each files of lab which contains the readme file and gives information about how to run the program. Thank you.